APLIKACJE WEBOWE

Node.js + Express

Architektura · event loop · npm

routing · middleware

L4

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Ten sam język, dwa światy

Przeglądarka

Frontend — to, co widzi użytkownik

- window, document, DOM
- Kliknięcia, formularze, animacje
- Brak dostępu do plików i systemu
- Kod widoczny dla każdego

Node.js

Backend — serwer, którego nie widać

- fs, http, process, os
- Pliki, sieć, bazy danych
- Obsługuje wielu klientów naraz
- Kod ukryty na serwerze



Node.js to środowisko uruchomieniowe

Nie język. Nie framework. Program, który wykonuje JavaScript poza przeglądarką.

- **2009** Ryan Dahl łączy silnik V8 z Chrome z biblioteką do asynchronicznego I/O
- **Cel** Serwery, które obsłużą tysiące połączeń bez tysięcy wątków
- **Dziś** API, czaty, narzędzia (npm, Vite, VS Code), aplikacje desktop

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

```
$ node -v
$ node 01-node-podstawy/a-hello.js Taylor
```

- typeof window → undefined: nie ma przeglądarki
- require('os'): dostęp do procesora i pamięci
- process.argv: argumenty z linii poleceń



Co jest w środku Node.js



by **Mateusz Węglewski**

Jeden kelner, wiele stolików

Model blokujący

Kelner przyjmuje zamówienie i stoi w kuchni, aż danie będzie gotowe. Więcej gości = więcej kelnerów (wątków).

Model Node.js

Kelner oddaje zamówienie kuchni i idzie do kolejnego stolika. Kuchnia woła, gdy danie gotowe — to jest callback.

Kelner = wątek JS · kuchnia = libuv i system · „ready!” = zdarzenie

by ***Mateusz Węglewski***

Fazy event loop

timers	pending	poll	check	close
setTimeout, setInterval	odłożone błędy systemowe	I/O: pliki, sieć, żądania HTTP	setImmediate	zamknięcie połączeń

↻ powtarzaj, dopóki są zadania — potem proces się kończy

Mikrozadania — mają pierwszeństwo
Po kodzie synchronicznym i po każdym callbacku: najpierw process.nextTick, potem Promise.then / await

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Zgadnij kolejność

```
console.log('A');
setTimeout(() => console.log('B'), 0);
setImmediate(() => console.log('C'));
fs.readFile(__filename, () => console.log('D'));
Promise.resolve().then(() => console.log('E'));
process.nextTick(() => console.log('F'));
console.log('G');
```

```
$ node b-event-loop.js
```






APLIKACJE WEBOWE

Blokowanie serwera

```
$ node c-blokowanie.js
```

#	Karta przeglądarki	Co się dzieje
1	/szybko	Odpowiedź natychmiast
2	/wolno + /szybko	Obie czekają ~5 s — pętla stoi
3	/wolno-async + /szybko	/szybko od razu, bez czekania





APLIKACJE WEBOWE

NIE BLOKUJ PĘTLI ZDARZEŃ.

- Czekanie na I/O — asynchronicznie (callback, Promise, await).
- Ciężkie obliczenia — poza głównym wątkiem.



npm = Node Package Manager

Nie piszemy wszystkiego od zera — pobieramy gotowe paczki.
npm to trzy rzeczy naraz:

Rejestr

npmjs.com — ponad 3 mln publicznych paczek

Narzędzie CLI

Komenda npm instalowana razem z Node

package.json

Opis projektu: nazwa, skrypty, zależności

by ***Mateusz Węglewski***

Anatomia package.json

```
{
  "name": "api-szkoly",
  "version": "1.0.0",
  "scripts": {
    "start": "node app.js",
    "dev": "node --watch app.js"
  },
  "dependencies": {
    "express": "^5.2.1"
  }
}
```

name, version
Tożsamość projektu

scripts
Skróty: npm start, npm run dev

dependencies
Czego projekt potrzebuje do działania

devDependencies
Tylko do pracy: testy, lintery

by *Mateusz Węglewski*



Komenda	Co robi
npm init -y	Tworzy package.json
npm install express	Dodaje paczkę do dependencies
npm i -D nazwa	Dodaje do devDependencies
npm install	Instaluje wszystko z package.json
npm start / npm run dev	Uruchamia skrypt ze scripts
npx nazwa	Uruchamia narzędzie bez instalacji



Wersjonowanie semantyczne

^5.2.1

MAJOR
Zmiany łamiące zgodność

MINOR
Nowe funkcje, zgodne wstecz

PATCH
Poprawki błędów

^5.2.1 — dowolna 5.x.x ≥ 5.2.1 · ~5.2.1 — tylko 5.2.x · 5.2.1 — dokładnie ta

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Projekt od zera w 60 sekund

```
$ mkdir moj-serwer && cd moj-serwer
$ npm init -y
$ npm install express
$ ls # package.json, package-lock.json, node_modules/
```




Po co nam Express?

Czysty HTTP

```
if (req.method === 'GET' &&
  /^\/uczniowie\/\d+$/
    .test(url.pathname)) {
  const id = Number(
    url.pathname.split('/')[2]);
  res.writeHead(200, {
    'Content-Type':
    'application/json' });
  res.end(JSON.stringify(u));
}
```

Express

```
app.get('/uczniowie/:id',
  (req, res) => {
    res.json(u);
  });
```

Minimalny framework na module http: routing, parametry, JSON, middleware. Nie narzuca struktury.

by ***Mateusz Węglewski***

Najmniejsza aplikacja Express

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Cześć!');
});

app.listen(3000, () =>
  console.log('http://localhost:3000'));
```

1. Import i utworzenie aplikacji
2. Trasa: metoda + ścieżka + funkcja
3. Nasłuchiwanie na porcie

req — co przyszło od klienta
res — co odsyłamy

by ***Mateusz Węglewski***

Trasa = metoda + ścieżka

Trasa	Znaczenie (REST)
GET /api/uczniowie	Pobierz listę
GET /api/uczniowie/:id	Pobierz jednego
POST /api/uczniowie	Dodaj nowego (dane w body)
PUT /api/uczniowie/:id	Zaktualizuj — zadanie na lab
DELETE /api/uczniowie/:id	Usuń

`req.params`
/uczniowie/**7** → id: '7'

`req.query`
?**klasa=4TI** → klasa: '4TI'

`req.body`
JSON z POST (po express.json)

by ***Mateusz Węglewski***

express.Router — trasy w osobnych plikach

routes/uczniowie.js

```
const router = express.Router();

router.get('/', (req, res) ⇒ {
  res.json(uczniowie);
});
router.get('/:id', ...);
router.post('/', ...);

module.exports = router;
```

app.js

```
const uczniowie =
  require('./routes/uczniowie');

app.use('/api/uczniowie', uczniowie);
```

Prefiks z app.use + ścieżka z routera:
'/api/uczniowie' + '/:id'

by ***Mateusz Węglewski***

Żądanie przechodzi przez łańcuch




```
function (req, res, next) { ... }
```

Każde ogniwo może: **odczytać lub zmienić** req i res · **zakończyć** odpowiedź (res.send) · albo **przekazać dalej** — next()

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

Rodzaje middleware

Rodzaj	Przykład	Działa dla
Wbudowane	express.json(), express.static()	Wszystkich żądań
Zewnętrzne (npm)	cors(), morgan(), helmet()	Wszystkich żądań
Własne globalne	app.use(logger)	Wszystkich żądań
Na trasie	router.post('/', wymagajKlucza, ...)	Tylko tej trasy
Obsługi błędów	(err, req, res, next)	Gdy coś wybuchnie

Middleware błędów ma **4 argumenty**, explanation in two lectures..





APLIKACJE WEBOWE

Własny logger — i co, gdy zapomnisz next()

```
function logger(req, res, next) {
  const start = Date.now();
  res.on('finish', () ⇒ {
    console.log(req.method, req.originalUrl,
      res.statusCode, Date.now() - start);
  });
  next(); // ← zakomentuj!
}
```

```
$ npm run dev
```




Chroniona trasa POST

APLIKACJE WEBOWE

```
02-express-app/
├── app.js
├── routes/
│   └── uczniowie.js
├── middleware/
│   ├── logger.js
│   └── wymagajKlucza.js
├── public/index.html
└── package.json
```

Bez nagłówka x-api-key

```
401 Brak lub zły klucz API
```

Z nagłówkiem x-api-key: tajne123

```
201 {"id":3,"imie":"Ola"...}
```

Narzędzie: Thunder Client w VS Code, Postman lub curl

by ***Mateusz Węglewski***



APLIKACJE WEBOWE

:wq

L4



by ***Mateusz Węglewski***